

A Beginner’s Primer to the Four ML Architectures for the FXT Centrality Paper

Internal note for Mathias Labonté

May 27, 2026

How to read this document

You know the physics already. This note is a friendly textbook-style introduction to the four neural-network architectures we plan to train for the FXT centrality paper. By the last page you should be able to sit at a group meeting, hear someone say “we ran a DeepSets model against a Set Transformer with an evidential head,” and translate that into something concrete. We assume you have never built a neural network. We do not assume you remember any ML jargon, and we will define each term once when it first appears.

1 The ML basics, in two pages

1.1 What “training a neural network” actually means

A neural network is a parameterized function $f_\theta(x)$. The vector θ is a list of numbers (the *weights*, sometimes millions of them) that we are free to choose. The function takes some input x (in our case, features of a heavy-ion event) and returns a prediction (in our case, the impact parameter b and the centrality percentile).

The shape of f_θ is built out of two simple operations applied in alternation: a linear transformation $y = Wx + c$, and a fixed nonlinear “activation” applied element-wise (for example, ReLU, defined as $\max(0, z)$). Stack a few dozen of these layers and the overall function becomes flexible enough to approximate essentially any smooth mapping. Importantly, the network is just arithmetic — no mystery, no hidden physics.

Training is the process of choosing θ so that the network’s predictions match the truth on a set of events for which we know the answer. We measure the mismatch with a *loss function* $L(\theta)$, for example the mean squared error $L = \langle (b_{\text{pred}} - b_{\text{true}})^2 \rangle$. Then we adjust θ in the direction that decreases L . The directional derivative $\partial L / \partial \theta$ is the *gradient*, and the iterative recipe

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

is *gradient descent*. The number η is the *learning rate*, typically 10^{-3} to 10^{-4} . In practice we average the gradient over a small batch of events (say 256) at a time; this is *stochastic gradient descent* and is essentially the entire training algorithm.

To avoid the model memorizing the training set, we split our 350k SMASH events into three disjoint pieces: **train** (80 %, used to update weights), **validation** (10 %, used to pick hyperparameters and

decide when to stop), and **test** (10 %, touched exactly once, at the very end, to report performance). Mixing these up is the single most common methodological mistake in applied ML, and we go to lengths to prevent it.

1.2 Permutation invariance, in physics language

A heavy-ion event is fundamentally a *set* of final-state particles. There is no first particle, second particle, third particle. If I reshuffle the list, the event is the same event. Any function we use to extract b from the particle list had better have the same property:

$$f(p_1, p_2, \dots, p_N) = f(p_{\pi(1)}, p_{\pi(2)}, \dots, p_{\pi(N)})$$

for any permutation π . This is *permutation invariance*, and it is the central inductive bias in three of our four architectures. “Inductive bias” just means a structural assumption baked into the model so it does not have to learn that assumption from data.

A vanilla MLP (multi-layer perceptron) given a flat vector of particle 4-momenta does *not* have this property: swap particles 7 and 8, and the inputs to all downstream neurons change. Three of our four architectures fix this by construction. The fourth (the MLP baseline) sidesteps it by feeding only permutation-invariant *summary* features (counts, means, sums).

A useful analogy: computing the total charge of a system from a list of constituents is automatically permutation-invariant, because summation is. Computing the total p_T is the same. So is the mean rapidity. These are all examples of the same universal recipe: particle-wise function, then symmetric pool, then a function of the pool. This is exactly the DeepSets template (Sec. 3).

1.3 What the four models share

By design (see `project_scope_v2.md`, locked 2026-05-17), all four architectures see the same inputs. This is so that the only difference between them is the inductive bias, not the data.

- **Per-particle features** (variable length, one row per charged hadron passing $0 \leq \eta_{\text{lab}} \leq 1.5$ ¹): p_T , η_{lab} , ϕ , charge.² No PDG, no mass — we deliberately exclude particle ID at the centrality stage, so the method does not depend on PID performance. At $\sqrt{s_{NN}} = 3.5 \text{ GeV}$ the most central 5 % of events have a median of ~ 150 such tracks in this acceptance (measured from `data/processed/cached/all_lab_truth.h5`, top-5 % by truth b , $0 \leq \eta_{\text{lab}} \leq 1.5$); a peripheral event can have zero.
- **Event-level features** (always four scalars per event): $\sqrt{s_{NN}}$, multiplicity in the acceptance window (mult_{lab}), mean p_T , and total p_T . $\sqrt{s_{NN}}$ is what makes joint multi-energy training possible.

The MLP uses only the event-level features (plus a handful of derived summaries). The other three architectures see the full particle list *and* the event-level features, the latter concatenated after the permutation-invariant pool.

¹The project scope memo records the wider window $0 < \eta_{\text{lab}} < 2$; the current code uses $0 \leq \eta_{\text{lab}} \leq 1.5$ per STAR FXT TPC (Kimelman thesis); this discrepancy is to be reconciled.

²Frame convention: SMASH and UrQMD generate events in collider (CM) mode with two opposing beams. For FXT-faithful analysis we Lorentz-boost into the target-rest frame using $y_{\text{boost}} = \text{arccosh}(\sqrt{s_{NN}}/2m_N)$ along the beam axis; η_{lab} denotes pseudorapidity in that boosted frame and matches the geometry STAR uses at fixed-target energies. The boost is implemented in `scripts/build_padded_cache.py`. (Note: a parallel CM-frame pipeline also exists in the codebase for cross-checks; the primer’s quoted numbers are lab-frame unless stated otherwise.)

1.4 What the four models output (the heads)

All four models share the same two output heads:

1. An **evidential regression head** for b . Instead of outputting a single number, it outputs four numbers $(\mu, \nu, \alpha, \beta)$ that parameterize a Normal-Inverse-Gamma (NIG) distribution. The practical consequence is that every prediction comes with its own uncertainty bar — one number for the best-estimate b and another for “how sure the model is.” This is what lets us draw a coverage plot later. (Caveat: a recent paper [Amini-NIG “mirage,” arXiv:2510.18322] argues NIG uncertainty can be miscalibrated; we will cross-check with deep ensembles for the headline result.)
2. A **centrality percentile head**. A softmax classifier over 10 or 20 bins (0–5 %, 5–10 %, ...). The softmax produces a probability for each bin, which we read either as a hard prediction (*argmax*) or as a soft distribution.

The total loss is a weighted sum of the regression loss (NIG negative log-likelihood plus a regularizer) and the cross-entropy loss for the classifier. We train both heads jointly.

2 Architecture 1 – MLP feature-vector baseline

2.1 Input

A flat vector of ~ 10 – 20 scalar event-level summary features: $\sqrt{s_{NN}}$, charged multiplicity in η_{lab} windows (several sub-windows: 0–0.5, 0–1, 0–1.5), mean p_T , total p_T , the ratio of forward to mid-rapidity multiplicity, the number of protons, the number of antiprotons, and a small handful of similar hand-built quantities. Crucially, *no* particle-list input.

2.2 Core idea

The MLP — multi-layer perceptron — is the workhorse of plain-vanilla deep learning. It is a stack of dense layers:

$$h_{\ell+1} = \sigma(W_{\ell}h_{\ell} + c_{\ell})$$

where σ is a nonlinearity (we use ReLU), W_{ℓ} is a weight matrix, c_{ℓ} a bias vector, and h_{ℓ} is the hidden representation at layer ℓ . We use 2–4 layers, widths of 128–512, and a small amount of *dropout* (randomly zeroing 10% of neurons during training) as a regularizer.

There is no per-particle reasoning here at all. The model sees the same RefMult observable a Glauber-RefMult fit uses, plus several additional event-level summary scalars — so strictly more information than a single-observable Glauber fit. The inductive bias is essentially zero — we are just giving the model a nice flexible parametric form that can learn the mapping (summary numbers) $\rightarrow$ (b, percentile) better than a hand-tuned threshold.

2.3 Physicist analogy

This is the ML equivalent of polynomial regression on engineered features. If you have ever fit a yield versus centrality with a cubic in mean p_T and multiplicity, you have already done this in spirit. The MLP just lets the polynomial be much higher order and much more flexible.

2.4 Why we picked it

It is the lower-bound baseline. The MLP tells us: *how much performance comes from having any flexible nonlinear model at all, vs. how much actually requires per-particle information?* If the MLP on summary statistics already matches the more sophisticated permutation-invariant models, then per-particle information is not the key ingredient at FXT, and the paper’s story shifts to “calibrated UQ + multi-energy training do the work, even on summary features.” If the MLP loses, we have quantified the value of going particle-level.

2.5 Known limitations

It can only ever exploit information present in the hand-built summary features. Any signal living in the joint distribution of individual particle kinematics (e.g. angular correlations, p_T spectrum shape) is invisible to it. At FXT energies, where the per-event statistics are small (of order 10^2 tracks even in central events) and every track potentially matters, this is a real handicap.

3 Architecture 2 – DeepSets

3.1 Input

The variable-length particle list directly: $\{(p_{\text{T}}^{(i)}, \eta_{\text{lab}}^{(i)}, \phi^{(i)}, q^{(i)})\}_{i=1}^N$ where N varies event by event. We pad shorter events to the same length with masked-out zeros for efficient batching, but the network ignores the padding.

3.2 Core idea

Zaheer et al. (NeurIPS 2017) proved a clean theorem: *any* permutation-invariant function of a set can be written in the form

$$f(\{x_1, \dots, x_N\}) = \rho\left(\sum_{i=1}^N \phi(x_i)\right)$$

where ϕ and ρ are arbitrary functions (in practice, both are MLPs). The model first applies the *same* small MLP ϕ to each particle independently, producing a per-particle embedding (say, a 64-dim vector). It then *pools* these embeddings by summing (or averaging, or max-pooling — any symmetric operation) across the list. The result is a single fixed-size event embedding, which is then passed through another MLP ρ to produce the output.

The sum step is what guarantees permutation invariance: $\sum_i$ does not care about the order of the summands.

3.3 Physicist analogy

You have computed countless event-summed observables already. The total transverse energy $\sum_i E_{\text{T}}^{(i)}$ is exactly a DeepSets-style sum with $\phi(p) = E_{\text{T}}(p)$ and $\rho(\cdot) = \text{identity}$. The flow vectors $Q_n = \sum_i e^{in\phi_i}$ are the same pattern with $\phi(p) = e^{in\phi_p}$. DeepSets says: *let the network choose ϕ and ρ for you, by gradient descent, optimized for the task you actually care about.* Instead of hand-designing an event-summed observable, you let the data tell you which 64 summed observables matter for predicting b .

3.4 Why we picked it

It is the cleanest permutation-invariant architecture and the natural “next step up” from the MLP. The comparison MLP-vs-DeepSets isolates the value of per-particle information *cleanly*, because DeepSets only differs from the MLP in having an actual per-particle stage. It is also fast, easy to train, and easy to interpret — you can inspect what ϕ has learned.

3.5 Known limitations

The pool is a bottleneck. Two events that have the same per-particle sum but different per-particle structure (for example, two narrow back-to-back jets vs. an isotropic distribution with the same total p_{T}) collapse to the same event embedding. The model has no way to capture interactions *between* particles, only their independent contributions. For our purposes this may not matter much — FXT events are not jet-like — but it is the theoretical ceiling on what DeepSets can do.

4 Architecture 3 – Set Transformer

4.1 Input

Same as DeepSets: the variable-length per-particle list.

4.2 Core idea

The Set Transformer (Lee et al., ICML 2019) replaces DeepSets’ independent per-particle embedding with one in which particles can *see each other* through attention. The headline operation is the *self-attention block*. For a list of N particles, each particle i updates its embedding by taking a weighted average of *all other* particles’ embeddings, where the weights are learned based on which pairs are most relevant:

$$h_i^{\text{new}} = \sum_{j=1}^N \alpha_{ij} V(x_j), \quad \alpha_{ij} = \frac{\exp(Q(x_i) \cdot K(x_j))}{\sum_k \exp(Q(x_i) \cdot K(x_k))}.$$

The functions Q , K , V (“query, key, value”) are learned linear maps. After several such blocks, the per-particle embeddings encode not just the particle’s own properties but its relationship to every other particle in the event. The model finally pools using *PMA* (pooling by multi-head attention), which is itself a learned permutation-invariant operation.

The whole thing is still strictly permutation invariant, because attention is computed by a sum that does not depend on particle order — only the values of Q, K, V matter, which are particle-local quantities.

4.3 Physicist analogy

Imagine you wanted to estimate centrality not just from each particle’s p_T in isolation, but from the spatial pattern of particles relative to each other (the gradient of multiplicity in ϕ , two-particle correlations, the presence of a recoil jet, the azimuthal asymmetry that signals elliptic flow). Each of these requires comparing particles against other particles. Attention is the machinery that lets the network learn which pairs of particles matter for the task.

The cost: self-attention scales as $\mathcal{O}(N^2)$ in particles per event, which is fine for $N \sim 100$ FXT events but expensive for $N \sim 1000+$.

4.4 Why we picked it

Two reasons. First, it is the strongest permutation-invariant baseline in modern ML — if particle-particle correlations carry real centrality signal, this is the architecture that will find it. Second, the comparison Set Transformer vs. DeepSets isolates the value of *interactions* beyond simple sums. If they tie, volume-fluctuation signal at FXT lives in the marginals, not in correlations. If the Set Transformer wins, correlations matter, and that itself is a publishable physics finding.

4.5 Known limitations

More parameters, slower to train, slightly more sensitive to hyperparameters than DeepSets. The interpretability is worse: inspecting what attention has learned is a research topic in its own right. There is also a small risk of overfitting on small per-event N because attention has high capacity relative to the data per event.

5 Architecture 4 – Energy Flow Network (EFN)

5.1 Input

Same per-particle list as DeepSets and Set Transformer, but EFN distinguishes the “energy” (intensity) feature from the “angle” (direction) features. In our setup, p_T plays the role of energy, and $(\eta_{\text{ab}}, \phi, q)$ play the role of angles. (Charge isn’t an “angle” geometrically, but functionally it behaves the same way: a non-energy attribute of the particle.)

5.2 Core idea

The EFN (Komiske, Metodiev, Thaler — JHEP 2019) is a *constrained* DeepSets. Its functional form is

$$\text{EFN}(\{p_i\}) = F\left(\sum_{i=1}^N z_i \Phi(\hat{n}_i)\right),$$

where $z_i = p_T^{(i)}$ is the per-particle energy weight, $\hat{n}_i$ are the angular variables, Φ is a small MLP from angles to a latent vector, and F is a larger MLP from the pooled vector to the output.

Compare to the DeepSets form, $\rho(\sum_i \phi(p_i))$: the EFN takes $\phi(p_i) = z_i \Phi(\hat{n}_i)$, i.e. the per-particle embedding must be linear in the energy. That linearity constraint is the key. It makes the network *IRC-safe*: infrared-and-collinear safe. In jet-physics language, IRC safety means the output is continuous under (i) the addition of a zero-energy particle (infrared safety) and (ii) the exact collinear splitting of one particle into two particles sharing the same total energy (collinear safety). Strictly, IRC safety guarantees only these limits — it does *not* by itself guarantee robustness to a finite $p_T > 50$ MeV cut or to finite-width detector smearing. Separately from IRC safety, the EFN output is a continuous (smooth MLP) function of the per-particle p_T values, so small numerical perturbations in p_T produce small changes in output; this is the property we actually exploit in the Phase B detector-emulation comparison.

5.3 Physicist analogy

The most natural example is the standard jet observable $\sum_i p_T^{(i)} f(\eta_i, \phi_i)$ — this is exactly EFN form, with the function f replaced by a learnable MLP Φ . If you have used jet substructure observables like N -subjettiness, you have used an IRC-safe construction in spirit. EFN is the deep learning version: a function in this safe family, optimized end-to-end.

5.4 Why we picked it

Three reasons.

1. It is the standard permutation-invariant architecture in experimental HEP. Reviewers (especially LHC-side machine-learners) will ask “did you benchmark against EFN” — we want a yes.
2. IRC safety is a useful property for Phase B of our project, when detector emulation kicks in. EFN by construction should degrade gracefully under the $\sim 1\%$ p_T smearing and tracking inefficiency we apply. If it does (and DeepSets/Set Transformer do not), that is informative.
3. It positions us against the CBM PointNet line. PointNet is *not* IRC safe; using EFN explicitly contrasts our methodology with theirs.

5.5 Known limitations

The linearity-in-energy constraint costs some flexibility. If the optimal solution requires nonlinear weighting of particle energies (say, particles below a threshold should be ignored entirely), EFN cannot represent that exactly. Komiske et al. also introduced *PFN*, an unconstrained variant; we use the constrained EFN because the constraint is the whole point. Also, the choice of “energy” weight z_i is itself a modelling decision: the original EFN paper uses $z_i = p_T^{(i)}/p_T^{\text{jet}}$ or E_i/E^{jet} , and we use unnormalized $z_i = p_T^{(i)}$. That is a valid choice but it does mean the IRC-safety statement is relative to “soft in p_T ,” not “soft in total energy.” At FXT energies this distinction matters less than at the LHC because the particles are non-relativistic enough that p_T , E , and p_T/M ordering of particles is similar; we still expect the structural inductive bias to be useful.

6 Side-by-side comparison

	MLP	DeepSets	Set Transformer	EFN
Input type	Scalar summary features (~ 10 – 20 numbers)	Variable-length particle list	Variable-length particle list	Particle list, energy + angle split
Inductive bias	None beyond smoothness of the function	Permutation invariance via symmetric sum/mean pool	Permutation invariance + learned particle–particle interactions	Permutation invariance + linearity in energy (IRC safe)
Rough parameter count	10^4 – 10^5	10^5 – 10^6	10^6 – 10^7	10^5 – 10^6
Expected strength at FXT	Trivially fast; transparent; lower-bound baseline	Best simplicity-to-power ratio; clean comparison to MLP	Captures correlations if they matter (flow, jet-like patterns)	Robust to detector smearing; HEP-standard
Expected weakness at FXT	Blind to per-particle structure; can only learn from hand-engineered summaries	Pool is a bottleneck; no particle–particle interactions	Most parameters; risk of overfitting on small N ; harder to interpret	Linearity-in-energy constraint may cost flexibility; IRC safety less natural at FXT than for jets

Table 1: The four architectures at a glance. “Rough parameter count” is the order of magnitude we expect at the configurations we plan to train; final values depend on width and depth, which are hyperparameters.

7 What we expect to learn from running all four

Each pairwise comparison answers a distinct physics-or-method question.

- **MLP vs. DeepSets** — does per-particle information beat hand-engineered summary features at FXT? At top RHIC and LHC the answer is unambiguously yes. At low multiplicity it is genuinely open: a 100-track central event has very few moments that are not already captured by mean, sum, and a couple of η -window multiplicities.

- **DeepSets vs. Set Transformer** — do particle–particle correlations carry centrality signal beyond the marginals? If the Set Transformer wins by a noticeable margin, that is itself a finding worth a paragraph in the discussion section, because it tells the cumulant-analysis community something about what they are throwing away when they collapse to multiplicity.
- **DeepSets vs. EFN** — does the IRC-safety constraint cost us performance at truth level, and does it pay off under detector emulation in Phase B? The Phase A vs. Phase B comparison for EFN is one of the cleanest experiments in the project.
- **All four vs. Glauber/RefMult baselines** — this is the headline plot. We expect every ML model to beat the classical baselines in the peripheral regime where $\sim 47\%$ of FXT events have zero charged particles in $|\eta_{\text{CM}}| < 0.5$ (see CLAUDE.md; SMASH dumps CM-frame 4-momenta, so this is the CM-frame mid-rapidity acceptance). The interesting question is how the four stratify among themselves.
- **Calibrated UQ** — across the four architectures, are the evidential uncertainty bars actually well-calibrated? If the Amini-NIG “mirage” critique bites, we will see it as systematic over- or under-coverage, and we will fall back to deep ensembles for the headline numbers.

The four-architecture comparison is not just hedging: it is a controlled experiment in which the inputs and the heads are held fixed, and only the inductive bias varies. That is what makes the result publishable as a methodology study rather than as a one-off benchmark.

Further reading

- Zaheer et al., *Deep Sets*, NeurIPS 2017, [arXiv:1703.06114](#).
- Lee et al., *Set Transformer*, ICML 2019, [arXiv:1810.00825](#).
- Komiske, Metodiev, Thaler, *Energy Flow Networks: Deep Sets for Particle Jets*, JHEP 01 (2019) 121, [arXiv:1810.05165](#).
- Amini et al., *Deep Evidential Regression*, NeurIPS 2020, [arXiv:1910.02600](#).
- The Amini-NIG critique: [arXiv:2510.18322](#) — background reading on why we back up evidential UQ with deep ensembles.
- Mallick, Tripathy, Behera, et al., *Estimation of impact parameter and transverse sphericity in heavy-ion collisions using machine learning*, [arXiv:2103.01736](#) — the closest prior art at top RHIC/LHC.
- Omana Kuttan et al., *A fast centrality-meter for the CBM experiment*, Phys. Lett. B 811 (2020) 135872, [arXiv:2009.01584](#) — the closest prior art in the FXT regime, using PointNet.